
Unidad 1: Arquitecturas y herramientas de programación web

Índice

Unidad 1: Arquitecturas y herramientas de programación web.....	1
1 Tecnologías web.....	1
HTML, CSS y JavaScript.....	1
Protocolo http.....	2
2 Front-end, back-end y full-stack.....	3
3 Servidores web y servidores de aplicaciones.....	5
¿Qué es un servidor web?.....	5
¿Qué es un servidor de aplicaciones.....	5
¿Qué es la web stack?.....	5
Ejemplos de software concretos.....	6
4 API y Servicios web.....	6
¿Qué es una API?.....	6
¿Qué significa API?.....	6
¿Cómo funcionan las API?.....	6
Servicio web.....	7
5 Lenguajes de programación en entornos de servidor.....	7
PHP.....	8
JSP / Servlets.....	9
Python.....	9
JavaScript- Node.js.....	10
Ruby.....	10
Go (Goland).....	10
6 Herramientas para crear aplicaciones web.....	10
Editores y IDE de software.....	10
Librería o biblioteca de código.....	11
Frameworks.....	12
Software para la gestión de proyectos.....	13
7 Entornos de desarrollo y entornos de producción.....	13
¿Qué es un entorno de desarrollo?.....	13
Niveles del entorno de desarrollo.....	14
8 DEVOPS.....	16
9 Fuentes consultadas:.....	16

1 Tecnologías web

HTML, CSS y JavaScript

Una aplicación web, es una aplicación creada usando como base el lenguaje HTML. Por lo tanto, se trata de una aplicación que se ejecuta en un navegador de Internet.

Las aplicaciones web utilizan lenguajes que apoyan la labor de HTML y que son traducibles por parte de los navegadores. Esencialmente las aplicaciones web actuales utilizan:

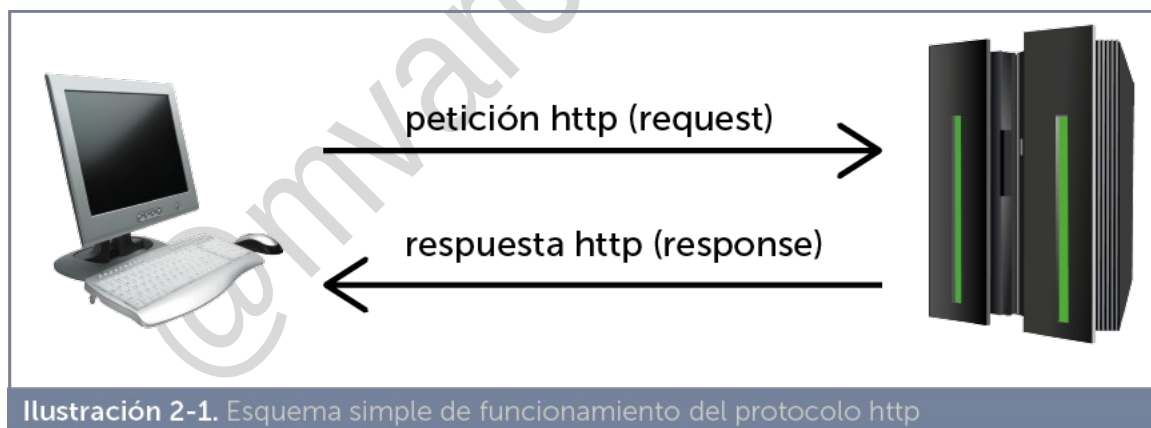
HTML. Para dar significado a los contenidos de la aplicación web. Permite indicar qué textos son títulos, cuáles son párrafos normales, cuáles son celdas de una tabla, cuáles son imágenes, etc.

CSS. Lenguaje que permite dar formato y maquetación a los contenidos. Color, tamaño de letra, posición, etc.

JavaScript. Permite diseñar la interactividad de la página. Permite que las acciones del usuario se puedan capturar y que la página reaccione a ellas. Ejemplos de sus posibilidades que cuando el usuario arrime el ratón a una imagen esta se ilumine, o que cuando el usuario haga clic en un título se muestre el contenido relacionado si estaba oculto o que el usuario pueda aumentar el tamaño de la letra de la página haciendo clic en un icono.

Protocolo http

La transmisión de páginas web (que en definitiva son documentos HTML) se realiza mediante el protocolo http, que es parte de la pila de protocolos TCP/IP en su capa de Aplicación. Se trata de un protocolo basado en una comunicación petición-respuesta; de modo que un cliente (también llamado *user agent*, agente de usuario) realiza una petición de recurso indicando su dirección en Internet. La petición llegará a un servidor http (también llamado servidor web), el cual responde a dicha petición, bien transmitiendo al cliente el recurso solicitado o bien indicando un mensaje de



error si el recurso no está disponible.

Los paquetes http que se intercambian entre el cliente y el servidor contienen una cabecera con información de control y luego el cuerpo. Este cuerpo es el código HTML en el caso de una página web, o la imagen, audio, etc. que se está transmitiendo.

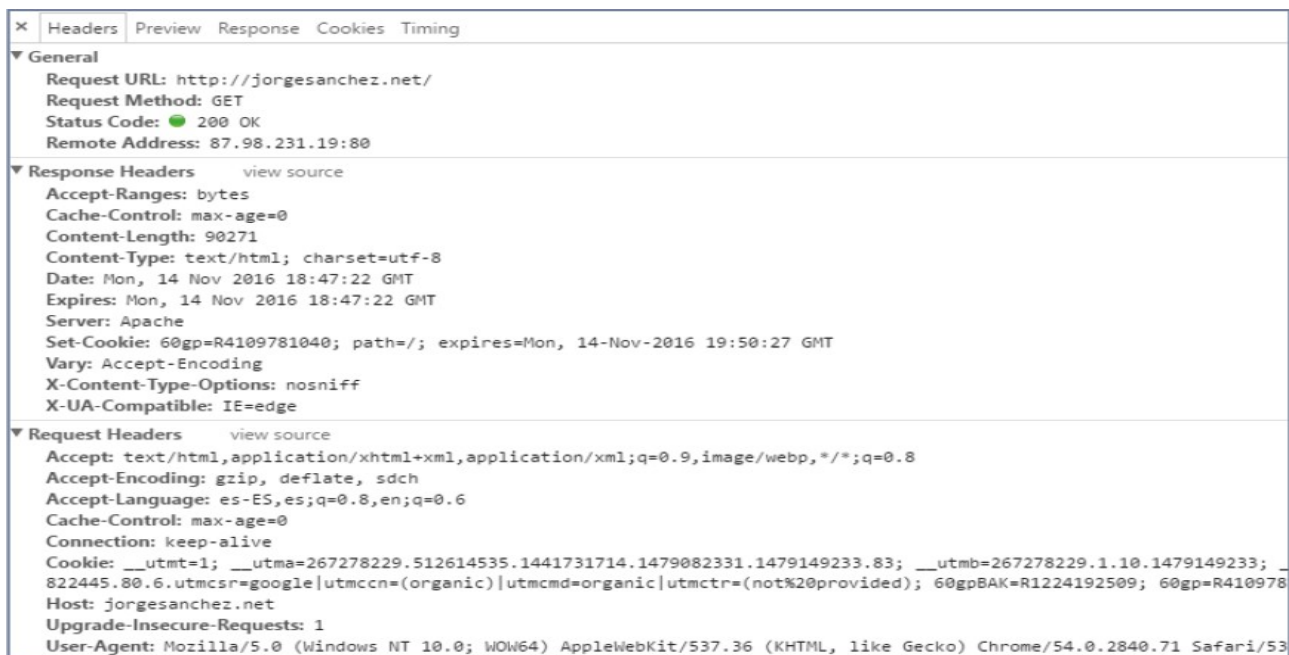


Ilustración 2-2. Cabecera que devuelve el paquete de respuesta http con la página de portada del sitio web jorgesanchez.net

En la Ilustración 2, se observa el paquete devuelto por el servidor web cuando recibe una petición http (de tipo GET) a la URL `http://jorgesanchez.net`. Se indica que la respuesta fue correcta (código 200, OK), la IP del servidor que responde, el tamaño del paquete, la fecha en la que expira, etc. Además se anexa la cabecera de la petición original (request headers) en la que se ve el navegador del usuario que hizo la petición, el lenguaje de su equipo, etc.

Además de la cabecera, el contenido de los paquetes http posee un cuerpo con el contenido del recurso que se solicitaba. Por ejemplo en el caso de haber solicitado una página web, el cuerpo contiene código HTML que el navegador podrá traducir.

El protocolo http sirve para transportar todo tipo de contenidos. De hecho, el código HTML de una página web hace referencia a otros archivos (imágenes, archivos CSS, etc.). Cuando el navegador detecta estas referencias, solicita al servidor web estos recursos. El navegador responderá con nuevos paquetes que contendrán (si está en su mano) el contenido solicitado.

El cliente (normalmente un navegador) será el encargado de mostrar el recurso siempre y cuando el navegador tenga la capacidad de hacerlo, ya que los navegadores pueden mostrar muchos formatos de contenido, pero por supuesto no todos.

Entre los recursos transmitidos por http que un navegador es capaz de reconocer están:

- Código HTML, CSS y JavaScript
- Imágenes JPEG, PNG, SVG y GIF
- Vídeos MP4 y otros formatos
- Audio MP3 y otros formatos
- Archivos XML (aunque no su validación ni el código XSL asociado)

2 Front-end, back-end y full-stack

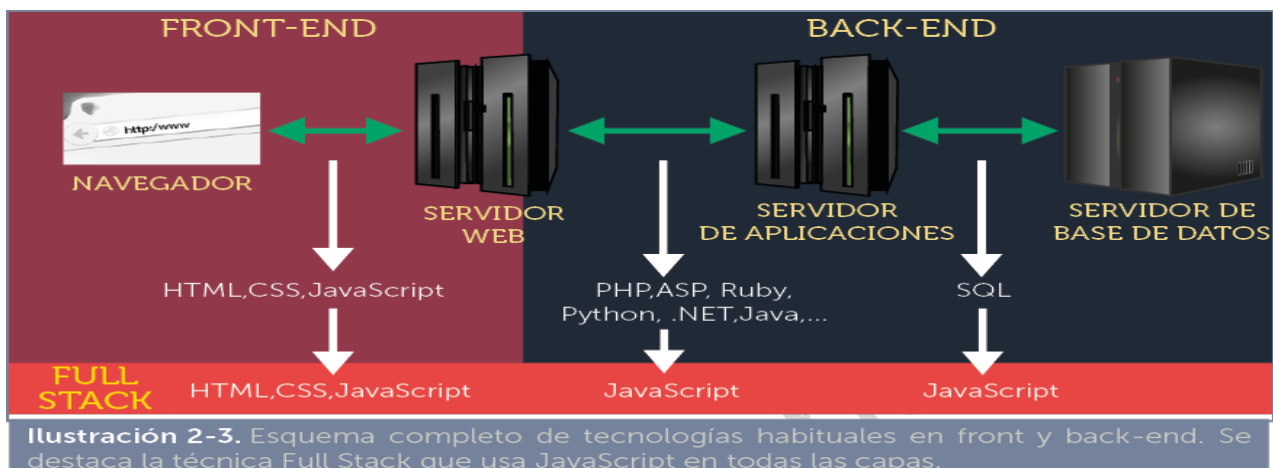
En este apartado se explica brevemente el funcionamiento de una aplicación web compleja. El único propósito es indicar que las aplicaciones web utilizan numerosas tecnologías.

En la mayoría de aplicaciones web complejas, cuando un navegador requiere de dicha aplicación, el servidor necesita ejecutar acciones como: **interpretar código del lado del servidor**

(por ejemplo en PHP o ASP.Net), pedir recursos a otros servidores (bases de datos, mapas, streaming de vídeo, etc.) o almacenar datos de sesión.

Estas acciones son totalmente opacas al usuario. Tras su ejecución, el navegador del usuario (user-agent) recibe código que sí es capaz de interpretar (normalmente HTML, CSS y JavaScript).

Esto implica que una aplicación web puede realizar tareas en el llamado **lado del servidor (server side)**. Es decir, tareas que se ejecutan en el servidor y cuyo código no podemos obtener. Por lo tanto, cuando se crea una aplicación web hay personas dedicadas a programar en código traducible en el lado del servidor y código en el lado del cliente,



El lado del servidor se asocia al desarrollo back-end, su programadores son denominados programadores back-end. Mientras que los profesionales encargados de la apariencia en el navegador, son llamados programadores front-end.

Desde un punto de vista de desarrollo una aproximación más detallada para este modelo de ejecución es lo que se conoce como modelo en 3 capas (MVC o model view controler). Es un modelo donde se muestra más en detalle como se distribuye el software que participa en cualquier desarrollo web.

- **Capa de presentación:** Es la capa donde la aplicación se muestra al usuario. Básicamente es la GUI (Graphical User Interface, Interfaz Gráfica de Usuario). En el caso de una aplicación web sería el código HTML que se carga directamente en el navegador web. En cualquier caso, se ejecuta directamente en el equipo del cliente.
- **Capa de negocio:** Es la capa intermedia donde se lleva a cabo toda la lógica de la aplicación. Siempre se ejecutará en el lado servidor. Esta capa, tras realizar todos los cálculos y/o operaciones sobre los datos, genera el código HTML que será presentado al usuario en la capa siguiente.
- **Capa de datos:** Es la capa que almacena los datos. Básicamente, en condiciones normales, hace referencia al propio SGBD que es el encargado de almacenar los datos. Dependiendo de la arquitectura de la aplicación, esta capa y la de negocio se pueden encontrar físicamente en el mismo equipo, aunque también es posible que se tengan que separar por cuestiones de rendimiento. La capa de datos sirve todas la información necesaria a la capa de negocio para llevar a cabo sus operaciones.

También teniendo en cuenta en que lado se ubican las tecnologías y para qué se utilizan éstas, actualmente se habla mucho de **3 perfiles diferenciados en el ámbito del desarrollo web**:

Front-end: Es la parte del desarrollo que se encarga del diseño y maquetación de la aplicación web utilizando tecnologías como HTML, CSS y Javascript (y sus frameworks). En este

caso debe preocuparse también de la correcta presentación en cualquier tipo de dispositivo e incluso del posicionamiento en buscadores

Back-end: Es la parte del desarrollo que se encarga del lado servidor utilizando tecnologías como PHP, JSP y Python. También se encarga de la administración del servidor de aplicaciones y la Base de Datos.

Full stack: En un perfil que engloba los dos anteriores. En este caso el desarrollador quizás no es un experto de ninguna tecnología concreta pero tiene amplios conocimientos de todo el conjunto y es capaz de colaborar en cualquiera de las partes.

3 Servidores web y servidores de aplicaciones.

Así como las aplicaciones de escritorio se ejecutan directamente sobre el propio Sistema Operativo, las páginas y aplicaciones web necesitan de una herramienta adicional que permita desplegarlas para su puesta en marcha. Hablamos de servidores web y servidores de aplicaciones, respectivamente.

¿Qué es un servidor web?

Un servidor web es una aplicación que recibe una petición HTTP (normalmente a través de un navegador web) y devuelve la página web solicitada (escrita en lenguaje HTML y pudiendo contener código Javascript inscrustado) para que ésta sea interpretada y visualizada por el navegador de quién realizó la solicitud (el usuario).

¿Qué es un servidor de aplicaciones

Un servidor de aplicaciones es una aplicación que contiene una serie de servicios los cuales están accesibles a través de una **API** expuesta a través de Internet. Normalmente los servidores de aplicaciones proporcionan más servicios que los servidores web. Por ejemplo, en el caso de los servidores de aplicaciones para Java o Python, éstos proporcionan un acceso transparente a la Base de Datos para que el desarrollador se centre exclusivamente en implementar la capa de negocio. Además, pueden proporcionar también servicios como fail-over o balanceo de carga.

¿Qué es la web stack?

Un stack de desarrollo web (web stack) es el grupo de herramientas y tecnologías que se usan en conjunto para desarrollar aplicaciones web. Es decir, son los lenguajes de programación, bibliotecas, frameworks, servidores y herramientas utilizados por los desarrolladores para llevar a cabo su cometido, la creación del software.

Un stack tecnológico o conjunto de soluciones **está conformado por varios elementos** que permiten que los programadores interactúen con una aplicación:

- Lenguajes de programación (ej. Python, Java, PHP)
- Frameworks de backend (ej. Laravel, Django, Ruby On Rails)
- Frameworks de frontend (ej. React, EmberJS, Bootstrap)
- Almacenamiento de datos (ej. MongoDB, MySQL, PostgreSQL)
- Servidores (ej. Apache, Ngnix, AWS, Azure)
- Monitoreo (ej. AppDynamics, Datadog)
- Soluciones de BI (ej. Tableau, Qlik, Power BI)
- API (ej. Mulesoft, Google Apigee)
- Herramientas de análisis (ej. Google Analytics, Heap)
- Soluciones UI / UX y patrones de diseño

Ejemplos de software concretos

Apache Web Server es uno de los servidores web más conocidos. Es software libre y multiplataforma, aunque aproximadamente el 90% de los servidores Apache se ejecutan actualmente en entornos Linux puesto que es el servidor preferido para esta plataforma.

Es muy modular lo que permite incorporar características una vez instalado y puesto en marcha. Eso le hace también muy flexible pudiendo dar servicio a webs escritas en los lenguajes de programación web más extendidos (como PHP, Python, ASP, . . .) a través del módulo correspondiente.

NGINX, pronunciado en inglés como «engine-ex», es un famoso software de servidor web de código abierto. En su versión inicial, funcionaba sólo como servidor web HTTP. Sin embargo, hoy en día también sirve como proxy inverso, balanceador de carga HTTP y proxy de correo electrónico para IMAP, POP3 y SMTP.

Apache Tomcat es un servidor de aplicaciones que funciona como contenedor de Servlets de Java. Actualmente es capaz de implementar varias especificaciones de Java EE como Servlets y JSP (Java Server Pages) y además proporciona un servidor web puro para que se use en combinación con el entorno Java.

WSGI (Web Server Gateway Interface)

WSGI es un interfaz que define como se comunica un servidor web con aplicaciones web o frameworks escritos en Python.

4 API y Servicios web

¿Qué es una API?

Las API son mecanismos que permiten a dos componentes de software comunicarse entre sí mediante un conjunto de definiciones y protocolos. Por ejemplo, el sistema de software del instituto de meteorología contiene datos meteorológicos diarios. La aplicación meteorológica de su teléfono “habla” con este sistema a través de las API y le muestra las actualizaciones meteorológicas diarias en su teléfono.

¿Qué significa API?

API significa “interfaz de programación de aplicaciones”. En el contexto de las API, la palabra aplicación se refiere a cualquier software con una función distinta. La interfaz puede considerarse como un contrato de servicio entre dos aplicaciones. Este contrato define cómo se comunican entre sí mediante solicitudes y respuestas. La documentación de su API contiene información sobre cómo los desarrolladores deben estructurar esas solicitudes y respuestas.

¿Cómo funcionan las API?

La arquitectura de las API suele explicarse en términos de cliente y servidor. La aplicación que envía la solicitud se llama cliente, y la que envía la respuesta se llama servidor. En el ejemplo del tiempo, la base de datos meteorológicos del instituto es el servidor y la aplicación móvil es el cliente.

Las API *pueden funcionar de cuatro maneras diferentes*, según el momento y el motivo de su creación.

API de SOAP

Estas API utilizan el protocolo simple de acceso a objetos. El cliente y el servidor intercambian mensajes mediante XML. Se trata de una API menos flexible que era más popular en el pasado.

API de RPC

Estas API se denominan llamadas a procedimientos remotos. El cliente completa una función (o procedimiento) en el servidor, y el servidor devuelve el resultado al cliente.

API de WebSocket

La API de WebSocket es otro desarrollo moderno de la API web que utiliza objetos JSON para transmitir datos. La API de WebSocket admite la **comunicación bidireccional** entre las aplicaciones cliente y el servidor. El servidor puede enviar mensajes de devolución de llamada a los clientes conectados, por lo que es más eficiente que la API de REST.

API de REST

Estas son las API más populares y flexibles que se encuentran en la web actualmente. El cliente envía las solicitudes al servidor como datos. El servidor utiliza esta entrada del cliente para iniciar funciones internas y devuelve los datos de salida al cliente. Veremos las API de REST con más detalle en otras unidades del temario.

Servicio web.

Un servicio web es una aplicación diseñada para establecer *comunicación con algún otro programa*, para eso se establecen *reglas de comunicación* como las direcciones para encontrar los recursos, qué acciones se pueden realizar con dichos recursos, y cómo se va a estructurar el texto que se intercambia entre ambos programas.

Un servicio web, no hace asunciones acerca de la tecnología que usarán los clientes para consumir la información, así, bien el servidor puede estar programado con JAVA u otro lenguaje de entorno de servidor y el cliente usar JavaScript, o cualquier otro tipo de tecnología. Es por esto que se establece un formato para que el texto intercambiado que sea estándar, podrá ser una estructura JSON, XML, ... entre otros.

La web funciona bajo una arquitectura cliente/servidor, donde el servidor es la o las computadora(s) que almacena la información y los recursos, mientras que el cliente es cualquier dispositivo que se conecte.

En términos prácticos, un servicio web está diseñado para intercambiar mensajes con otra aplicación, permitiéndonos enviar y recibir información.

Para establecer las reglas de comunicación, un servicio web puede usar algún protocolo de comunicación como graphql, REST, SOAP, etc. Estos protocolos establecen una normativa de cómo intercambiarán mensajes el cliente y el servidor. Una vez que se ha elegido el protocolo, los desarrolladores implementan sus aplicaciones siguiendo estas reglas para el envío y la lectura de mensajes.

5 Lenguajes de programación en entornos de servidor

Al principio, el éxito espectacular de la web tuvo lugar gracias al protocolo HTTP y al lenguaje HTML . El protocolo HTTP es una implementación fácil y sencilla de un sistema de comunicaciones que permite enviar cualquier tipo de archivos por la red. Este protocolo permite, además, atender a miles de peticiones reduciendo la potencia de los servidores y, como consecuencia, reduciendo los costes de despliegue de los servicios.

La web era un conjunto de páginas estáticas, documentos simples de información que podían consultarse o descargarse. Poco a poco fue evolucionando hasta permitir desarrollar páginas dinámicas que podían ser creadas en función de la petición enviada. Este método se conoció como CGI (Common Gateway Interface). Los CGI definen una comunicación entre el cliente y el servidor mediante HTTP en la que los clientes podían pedir información a los programas ejecutados en el servidor. CGI especifica un estándar para transferir datos entre el cliente y el programa. Es un mecanismo de comunicación entre el servidor Web y una aplicación externa. El resultado final de la ejecución son objetos MIME. Estas aplicaciones, que se ejecutan en el servidor, reciben el nombre de CGI.

Pero los CGI tienen un punto débil: cada vez que reciben una petición, el *servidor web comienza un proceso en el que ejecuta el programa CGI*. Como, por otra parte, la mayoría de CGI estaban escritos en algún lenguaje interpretado (Perl, Python, etc.), esto implica una gran carga para la máquina del servidor. Además, si la web tiene muchos accesos al CGI, esto supone graves problemas de rendimiento.

Por eso empiezan a desarrollarse alternativas a los CGI. Las soluciones consisten en dotar al servidor de un intérprete de algún lenguaje de programación (PHP, JSP, etc.) que permita incluir el código en las páginas de forma que el servidor sea quien lo ejecute, reduciendo así el tiempo de respuesta.

A partir de ese momento se vive una explosión del número de arquitecturas y lenguajes de programación que nos permiten desarrollar aplicaciones web. Surgen los lenguajes de programación integrados en el servidor que permiten interpretar instrucciones incrustadas en las páginas HTML y un sistema de ejecución de programas más enlazado con el servidor que no presenta problemas de rendimiento de los CGI.

PHP

PHP (PHP Hypertext Preprocessor) es un lenguaje de programación de lado servidor diseñado principalmente para el desarrollo web.

PHP se utiliza como ***lenguaje de script embebido en páginas HTML*** y ejecutado, normalmente, en un módulo del servidor web (por ejemplo, en Apache). El servidor web combina los resultados de ejecutar los scripts PHP con el HTML al que va embebido y genera la página web resultante para el navegador.

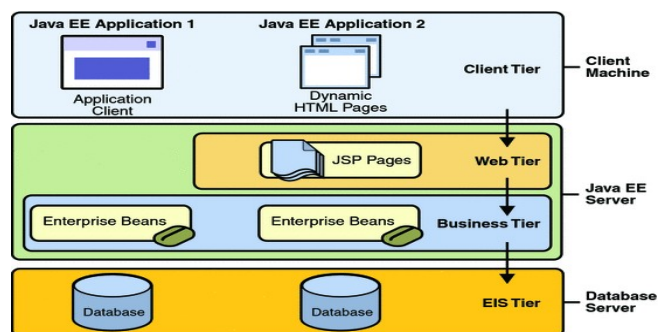
Actualmente PHP funciona prácticamente con cualquier servidor web y en cualquier Sistema Operativo existentes, y casi con cualquier SGBD en caso de que necesitemos utilizar una Base de Datos. A pesar de ello, lo más habitual es verlo formando lo que se conoce como una arquitectura LAMP (Linux, Apache, MySQL y PHP), es decir, funcionando sobre un Sistema Operativo Linux, ejecutándose como un módulo del servidor web Apache y utilizando a MySQL como SGBD para almacenar la información en caso de que se requiera una Base de Datos.

LAMP:



JSP / Servlets

JavaServer Pages (también conocido como JSP) es una tecnología creada para la creación de páginas web dinámicas del lado servidor. Al igual que PHP, su código se escribe embebido junto con el HTML de la página web y es el servidor de aplicaciones, en este caso, quién debe procesarlo para generar la página web resultante, en HTML.



Arquitectura habitual JSP/Servlets

Por otra parte, **Java Servlets** es una tecnología que también se puede utilizar para crear contenido web dinámico pero que además extiende su funcionalidad a la posibilidad de conectar esas webs dinámicas con otro contenido accesible a través de Internet. En ocasiones se utiliza junto con JSP para crear aplicaciones web más complejas.

Ambas son tecnologías desarrolladas por Sun Microsystems y propiedad ahora de Oracle, tras adquirir esta última a la primera hace ya algunos años.

Usualmente, las aplicaciones web realizadas mediante esta tecnología, suelen estar ubicadas en servidores TOMCAT

Python

El lenguaje Python fue diseñado por Guido van Rossum y ahora se mantiene gracias a una comunidad de desarrolladores. Es muy relevante dentro de la industria del software, completamente open source y capaz de llevar a cabo cualquier tipo de proyecto. Es usado en el ámbito de la web, para el desarrollo backend, pero también en el ámbito del cómputo científico, big data y educación.

Para su desarrollo en backend, es muy común, el uso de frameworks. Los ejemplos más relevantes de estos frameworks son:

- Django es uno de los frameworks más populares y ampliamente utilizado para el desarrollo web en Python. Es conocido por su enfoque en la simplicidad y la productividad, lo que lo convierte en una excelente opción para proyectos de todos los tamaños. Django proporciona muchas características incorporadas, como un sistema de administración, autenticación, ORM (Object-Relational Mapping).
- Flask es otro framework web muy popular en Python. Aunque es más minimalista que Django y no incluye tantas características integradas, esto permite una mayor flexibilidad y control sobre cómo estructurar tu aplicación. Flask es excelente para construir aplicaciones web pequeñas y medianas.

JavaScript- Node.js

JavaScript es uno de los lenguajes de programación más utilizados tanto en el frontend como en el backend. En el backend, **Node.js** es una plataforma que permite ejecutar JavaScript en el servidor, lo que ha ganado una gran popularidad en el desarrollo de aplicaciones web y API.

Ruby

Ruby on Rails es un framework web que utiliza el lenguaje Ruby. Es conocido por su simplicidad y productividad, lo que lo hace adecuado para el desarrollo rápido de aplicaciones web.

Go (Goland)

Go es un lenguaje desarrollado por Google que ha ganado popularidad en el desarrollo web debido a su rendimiento y eficiencia en aplicaciones concurrentes. Frameworks como Gin y Echo son populares en la comunidad de desarrollo Go.

6 Herramientas para crear aplicaciones web

Editores y IDE de software

Para escribir código en la mayor parte de los lenguajes de programación y en la mayoría de los lenguajes de marcas, bastaría con un editor de texto plano como el bloc de notas de Windows o vi de Linux/Unix. No obstante la escritura en este tipo de editores es incómoda ya que no proporcionan ayuda ni aceleran la escritura.

Además, los resultados se deben mostrar en un navegador, ejecutar en algún stack de servidor.

Para escribir el código lo ideal es trabajar mediante *editores de código capaces de entender el lenguaje y colorear* de diferente manera las etiquetas y palabras reservadas del lenguaje, para distinguirlas del texto normal. Además de que incorporen herramientas que faciliten la edición.

Así tenemos las siguientes opciones:

1. **Editores de texto multipropósito** (editores de código). Son programas que permiten editar texto, pero que están pensados para escribir código en cualquier lenguaje de programación. Su ventaja está en que colorean la sintaxis dependiendo del lenguaje en el que estamos escribiendo, porque son editores políglotas.

Cuando escribimos, colorean el texto resaltando el código de forma que sea más legible. Además nos ayudan a escribir código de forma más eficaz, porque incluso detectan los fallos en el mismo. Entre los más conocidos están Notepad++, TextMate, Geany, Atom, Visual Studio Code o Sublime Text.

2. **Editores de texto especializados** en escribir código en un lenguaje concreto. La diferencia respecto a los anteriores, es que, con estos, no podremos escribir código en lenguajes para los que no estén preparados, o si lo hacemos, no tendremos las ventajas que proporciona el

entorno. A cambio, tendremos más funciones para escribir en el lenguaje concreto para el que se han diseñado. Por ejemplo, podremos previsualizar el resultado del código HTML, (adelantándonos al navegador) hacer debugger de código y pruebas, coloreado más ajustado a los lenguajes, reconocimiento de librerías y plantillas de uso habitual de diseñadores,...

Algunos son: Coda, Komodo Edit, WebStorm o Brackets. Los editores Atom y Visual Studio Code hay quienes les colocan en este tipo porque, aunque son editores políglotas, están muy especializados en aplicaciones web.

3. **Entornos completos de programación.** Son los llamados IDE (Entornos de Desarrollo Integrado) que son entornos de trabajo para programar en lenguajes avanzados como Java o C++, pero que suelen trabajar perfectamente para crear proyectos para la web. Destacan Eclipse, Aptana (versión de Eclipse para lenguajes en la web), NetBeans, IntelliJ IDEA y Microsoft Visual Studio.
4. Editores online. Se trata de aplicaciones web que nos permiten escribir y probar nuestro código. Algunas de ellas, permiten probar ayuda de completitud de código con IA. Algunos ejemplos son:

http://www.w3schools.com/html/tryit.asp?filename=tryhtml_basic

<https://replit.com/>

<https://codepen.io>

5. **Editores visuales (Usualmente para client side).** Permiten trabajar de forma que podamos construir páginas web sin escribir código, de forma visual. Se basan en el software WYSIWYG (What You See Is What You Get), lo que se ven en pantalla es el resultado que se obtendrá. Referido a que trabajas viendo el trabajo tal cual quedará finalmente. En realidad es imposible ya que el aspecto final dependerá del dispositivo y software que se utilice al ver la página y eso variará enormemente de un usuario a otro; pero al menos permiten hacerse una idea muy visual del resultado.

Adobe Dreamweaver sería el más famoso de este tipo y el más completo, ya que también permite trabajar de forma cómodo con el código. Otras opciones son Adobe Muse, Tumult Hype, Google Web Designer, Kompozer, WYSIWYG Web Builder o Xara Web Designer.

Librería o biblioteca de código

Es un *conjunto de implementaciones codificadas* en un lenguaje de programación. Estas implementaciones *ofrecen una interfaz definida* para ciertas funcionalidades. Piensa en ellas como instrucciones que diferentes programas pueden usar. Por ejemplo, si estás haciendo una app para enviar correos automáticamente, puedes usar una librería para manejar las fechas. Esto ahorra tiempo y recursos al aprovechar código ya hecho. Es importante notar que las librerías se vinculan en distintas etapas del desarrollo y podrán ser privadas o públicas, estáticas o dinámicas.

Librerías estáticas. Suelen estar construidas en el mismo lenguaje en el que estamos programando, usualmente, se añaden a nuestro código en el proceso de linkado antes de hacer el ejecutable. Una vez compilado, forman parte de nuestro software como si fueran código implementado por nosotros.

Librerías dinámicas. Las librerías dinámicas, usualmente no pertenecen al proyecto, sino que se incorporan a partir de algún otro elemento en el stack de ejecución de una aplicación. Un ejemplo claro son las librerías del sistema operativo. Nuestro software hace uso de estas librerías (ejemplo abrir un archivo), pero el código pertenece al propio sistema operativo dónde estamos ejecutando nuestra aplicación.

Frameworks

¿Qué es un Framework?

Un framework es un entorno o marco de trabajo, un conjunto de prácticas, conceptos y criterios a seguir estandarizados. Siguiendo unas reglas, el framework nos obliga a utilizar buenas prácticas para nuestro código.

Por otro lado, los frameworks también nos **proporcionan una serie de herramientas ya desarrolladas**. Suelen ser funciones comunes en todos los proyectos. Por ejemplo, si tenemos un proyecto web seguramente tendremos usuarios y si tenemos usuarios lo más normal es que estos tengan que hacer «login». Esta funcionalidad de hacer «login» con un correo electrónico y una contraseña ya estaría hecha si utilizamos un framework.

Al final, el objetivo del framework es intentar ahorrarnos trabajo. Además, con una serie de clases y funciones ya listas para su uso.

Pero no solo esto. El framework también nos ayuda respecto al trabajo en equipo. Imagina un proyecto en el que trabajan varios programadores. Al final, el framework nos obliga a trabajar de una cierta manera para que todos los programadores trabajemos en la misma línea y sepamos cómo se están haciendo las cosas. En resumen, todo estará bajo la misma lógica del framework.

¿Para qué sirve un Framework?

En línea con el punto anterior, un framework es un conjunto de:

- Convenciones, estándares o paradigmas y buenas prácticas
- Funcionalidades costosas ya desarrolladas

Desarrollar software es un proceso complejo que requiere de una gran cantidad de tareas, incluida la codificación, el diseño y las pruebas. Solo para la parte de codificación, los programadores tienen que encargarse de la sintaxis, las declaraciones, excepciones, etc.

En este sentido, los software frameworks facilitan la vida a los desarrolladores al permitirles tomar el control de todo el proceso de desarrollo desde una única plataforma.

Sus **objetivos** son los siguientes:

- Servir como base para tus programas
- No reinventar la rueda de nuevo. Es decir, no estar programando una y mil veces.
- Evitar tareas repetitivas
- Aumentar la productividad ya que, al tener cosas ya hechas, nos centramos realmente en el problema que tenemos que resolver
- Favorecer el trabajo en equipo
- Infundirte buenas prácticas

Ventajas de usar Framework

Como hemos comentado anteriormente, a diferencia de muchas otras herramientas de desarrollo, los frameworks proporcionan una base de código predefinida y pautas para facilitar el desarrollo web sin problemas. Al mismo tiempo, acelera el proceso de desarrollo y reduce el tiempo de comercialización al **acortar el ciclo de vida del desarrollo de la aplicación**.

A continuación, señalamos **los principales beneficios** de usar frameworks:

Proceso de depuración más sencillo: El desarrollo de aplicaciones web utilizando frameworks se centra principalmente en mejorar la calidad y la legibilidad del código para la conveniencia de los desarrolladores. Además de proporcionar la forma más eficiente de codificar y desarrollar aplicaciones web también facilitan la depuración y el mantenimiento del software.

Eficiencia de código mejorada: El objetivo principal es mejorar la legibilidad y la reutilización del código, proporcionando así la forma más rápida y eficiente de desarrollar

aplicaciones web autosostenibles. Además, los desarrolladores obtienen un entorno de codificación receptivo y funciones avanzadas que conducen a un desarrollo web más rápido.

Fácil reutilización de código: Los desarrolladores no necesitan escribir estructuras de código complejas que contengan cientos de líneas de código. En su lugar, pueden utilizar la base de código predefinida y realizar modificaciones sencillas para poder trabajar sin problemas.

Desarrollo más rápida: Con una plantilla predefinida y una interfaz de codificación intuitiva, los desarrolladores pueden ahorrar horas de esfuerzo en codificación. Además, los frameworks ofrecen creación rápida de prototipos, depuración sencilla y cambios de código en tiempo real para implementar funcionalidades personalizadas.

Seguridad mejorada: Su uso para el desarrollo de aplicaciones web protege y salvaguarda su aplicación web contra ataques comunes como DDoS, inyección SQL y evita la manipulación de datos. Se debe a las funciones de seguridad integradas que la mayoría aportan.

Software para la gestión de proyectos.

El software de gestión de proyectos es una herramienta que ayuda a los equipos a organizar, dar seguimiento y ejecutar el trabajo. Básicamente, es como un espacio interactivo, donde coordinas la forma en que trabajas, colaboras y te comunicas con los demás. Pero la mejor solución de gestión de proyectos va mucho más allá: conecta tus flujos de trabajo más avanzados en toda la empresa y te ayuda a lograr tus objetivos más importantes y audaces.

Es una categoría amplia y puede ser tan simple como listas de verificación digitales o tan compleja como una plataforma de gestión del trabajo completa. El tipo de software de gestión de proyectos que elijas, y cómo lo uses, dependerá del tamaño, las necesidades y los objetivos generales de tu organización.

Ejemplos de este tipo de herramientas son:

- **Maven.** Es una herramienta de software para la gestión y construcción de proyectos Java creada por Jason van Zyl, de Sonatype, en 2002.
- **Apache Ant.** Es una herramienta usada en programación para la realización de tareas mecánicas y repetitivas.
- **Make (archivos makefile).** Es una herramienta de gestión de dependencias, típicamente, las que existen entre los archivos que componen el código fuente de un programa, para dirigir su recompilación o "generación" automáticamente.
- **Webpack** toma las dependencias y genera un gráfico de dependencia que permite a los desarrolladores de la web utilizar un enfoque modular para sus propósitos de desarrollo de aplicaciones web.

7 Entornos de desarrollo y entornos de producción.

El desarrollo de software puede llegar a ser una de las áreas más complejas a nivel tecnológico en una empresa. Es por ello que utilizar entornos de desarrollo adecuados puede ser la mejor opción para la creación y mejora de los diferentes programas. Estas herramientas permiten a los desarrolladores agilizar los procesos y tareas, para poder terminar de forma ágil y en menos tiempo grandes proyectos de desarrollo.

¿Qué es un entorno de desarrollo?

Un entorno de desarrollo en el mundo del software y la tecnología es un conjunto de procedimientos y herramientas utilizadas por los desarrolladores para codificar, generar, depurar, actualizar, integrar, testear, validar y ejecutar programas.

Funciona como un espacio de trabajo en el que los cambios se implementan en diferentes entornos hasta que se ponen en marcha en la versión real (versión del usuario). Actualmente nos referimos a un entorno de desarrollo al proceso integral de gestión del desarrollo de software.

Para qué sirve implementar un entorno de desarrollo

Básicamente, implementar un entorno de desarrollo sirve para facilitar el trabajo realizado por los desarrolladores, realizar un control de versiones, realizar pruebas y actualizar la documentación de un proyecto. Este tipo de metodología permite que el desarrollo de software se realice de forma ágil, acortando los tiempos.

Niveles del entorno de desarrollo

En el desarrollo de software existen diferentes entornos de desarrollo que se ejecutan a medida que va avanzando el proyecto.

Entorno de desarrollo

El entorno de desarrollo propiamente dicho se refiere a la creación del software. Este entorno puede ser manejado a nivel local en el ordenador del programador o en un servidor local. Así mismo, puede ser procesado en herramientas desarrolladas para este tipo de tareas.

En este espacio de trabajo el desarrollador codifica, realiza las pruebas iniciales y comprueba la ejecución del código.

Entorno de Integración

Es habitual que trabaje un equipo en el desarrollo de un proyecto, lo que hace que se realicen tareas de forma individual. El entorno de integración facilita la unión de los diferentes desarrollos y permite comprobar que los diferentes trabajos no interfieran entre sí.

Entorno de pruebas o testing

Este tipo de entorno usualmente está ubicado en la nube. La ventaja de contar con un servidor de pruebas es que permite a otros miembros del equipo y clientes a interactuar con el software para realizar tests de funcionalidades del sistema.

También se pueden realizar testing automatizados, su principal objetivo es detectar el mayor número de fallos posible, al no necesitar una persona que realice un testeo manual permite ahorrar tiempo. Además, la ejecución de pruebas automatizadas permite ejecutar tests durante las 24 horas del día los 7 días de la semana.

Entorno de pre-producción

También llamado entorno de staging, permite trabajar en un entorno con una configuración exactamente igual a la que existe en el entorno de producción. La finalidad de usar este tipo de servidor es simular el entorno de producción para validar su usabilidad en un entorno real. Del mismo modo permite el poder realizar pruebas de actualizaciones y minimizar las causas que puedan generar caídas del sistema.

Entorno de producción

Es el entorno donde finalmente se ejecuta el software, el cual es utilizado por el usuario. Este servidor, a diferencia del servidor de pre-producción, debería tener una mayor infraestructura y mayor capacidad de manejo de tráfico o de conexiones recurrentes.

Si el entorno de producción está bien configurado, se han realizado pruebas automatizadas y por parte del usuario. No debería existir ninguna incidencia en la ejecución del software final.

Entornos de desarrollo integrado en la nube

Trabajar con herramientas de entornos de desarrollo en la nube tiene muchos beneficios en comparación con los entornos locales.

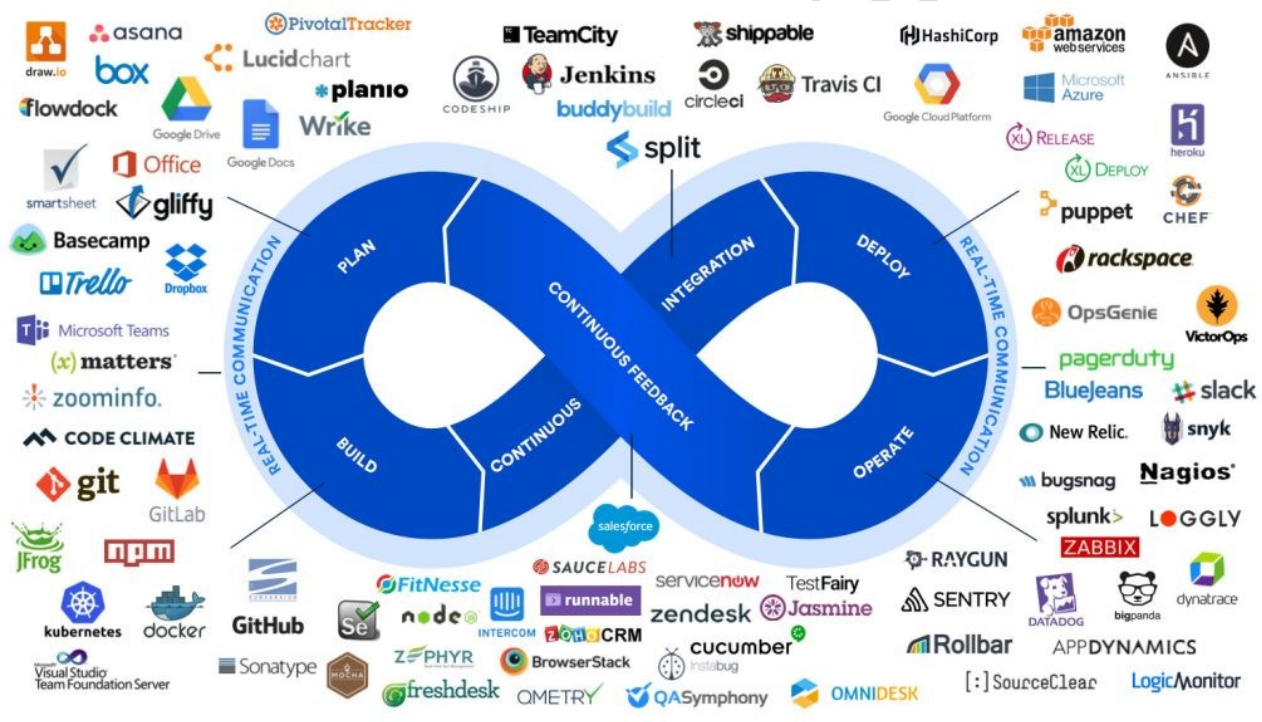
Cuando se trabaja con un entorno de desarrollo cloud no es necesario descargar el software, el sistema no consume una gran cantidad de recursos informáticos y no hay que realizar configuraciones locales. Todo esto tiene como consecuencia que el entorno de desarrollo se pueda utilizar de forma inmediata y los desarrolladores puedan colaborar en los diferentes proyectos de forma rápida.

Este tipo de implementación también influye en el equipo, ya que no es necesario que todos los desarrolladores se encuentren en el mismo lugar, sino que pueden contribuir desde diferentes puntos y dispositivos, algo que hoy en día se valora mucho debido a los cambios que ha traído el teletrabajo.

Además, los entornos en la nube permiten una gestión centralizada, ya que ningún código se encuentra en el ordenador de un desarrollador, ni hay diferentes versiones sin control pasando de una persona a otra.

Habitualmente, cuando se realiza una compilación, depuración o prueba en un entorno de desarrollo local, el ordenador queda inoperativo hasta que termine esa tarea. En cambio, con el almacenamiento cloud los procesos se pueden ejecutar sin monopolizar los recursos informáticos.

8 DEVOPS



El término "DevOps" surge de la unión de las palabras "development" (desarrollo) y "operations" (operaciones), pero representa un conjunto de ideas y prácticas que van más allá de la combinación de ambos conceptos. DevOps incluye sistemas de seguridad, maneras de trabajar en colaboración, análisis de datos, entre otras características. ¿Pero qué es?

DevOps describe los enfoques para agilizar los procesos con los que una idea (como una nueva función de software, una solicitud de mejora o una corrección de errores) pasa del desarrollo a la implementación en un entorno de producción en que puede generar valor para el usuario. Estos enfoques requieren que los equipos de desarrollo y operaciones se comuniquen con frecuencia y aborden su trabajo con empatía hacia sus compañeros de equipo. También es necesario que la preparación de los sistemas sea flexible y adaptable. Con DevOps, es posible brindar una mayor

potencia a quienes más la necesitan mediante el autoservicio y la automatización. Los desarrolladores, que generalmente realizan codificaciones en un entorno de desarrollo estándar, trabajan en estrecha colaboración con los equipos de operaciones de TI para agilizar el diseño, las pruebas y el lanzamiento de los sistemas de software, sin comprometer la confiabilidad.

Por supuesto, esto implica cambios más frecuentes en el código y el uso más dinámico de la infraestructura. Las estrategias tradicionales de gestión no pueden seguir el ritmo de la demanda, así que necesitará implementar algunas modificaciones para obtener la ventaja competitiva.

9 Fuentes consultadas:

<https://aws.amazon.com/es/what-is/api/>

<https://codigofacilito.com/>

<https://www.ekon.es/blog/entornos-desarrollo-software/>

<https://wikipedia.org>

<https://github.com/pekechis>